부분 다운로드 및 범위 요청 처리 분석

개요

iroh-blob은 대용량 파일의 효율적인 전송을 위해 부분 다운로드와 범위 요청 기능을 제공합니다. 이 시스템은 HTTP의 Range 요청과 유사하지만, BLAKE3 해시 기반의 검증과 청크 단위 처리를 통해 더욱 강력한 무결성 보장과 효율성을 제공합니다.

ChunkRanges를 이용한 범위 요청 구현

ChunkRanges 기본 구조

```
pub type ChunkRanges = RangeSet2<ChunkNum>;
```

ChunkRanges는 1024바이트 청크 단위로 데이터 범위를 표현하는 효율적인 자료구조입니다:

- **청크 기반 주소 지정**: 바이트 단위가 아닌 1024바이트 청크 단위로 범위를 지정
- **불연속 범위 지원**: 여러 개의 분리된 범위를 하나의 구조체로 표현
- **효율적인 집합 연산**: 합집합, 교집합, 차집합 등의 불린 연산 지원

범위 요청 생성 방법

1. 바이트 범위 요청

```
// 처음 1000바이트 요청
let request = GetRequest::builder()
.root(ChunkRanges::bytes(..1000))
```

```
.build(hash);


// 특정 바이트 범위 요청 (10000-11000바이트)
let  request = GetRequest::builder()
.root(ChunkRanges::bytes(10000..11000))
.build(hash);
```

2. 청크 범위 요청

```
// 처음 10개 청크 요청
let  request = GetRequest::builder()
.root(ChunkRanges::chunks(..10))
.build(hash);


// 특정 청크 범위 요청 (100-110번째 청크)
let  request = GetRequest::builder()
.root(ChunkRanges::chunks(100..110))
.build(hash);
```

3. 복합 범위 요청

```
// 여러 불연속 범위 동시 요청
let  request = GetRequest::builder()
.root(ChunkRanges::bytes(..1000) | ChunkRanges::bytes(10000..11000))
```

```
.build(hash);
```

ChunkRangesSeq를 통한 시퀀스 처리

ChunkRangesSeq는 해시 시퀀스의 각 블롭에 대한 범위를 효율적으로 표현합니다:

```
pub struct ChunkRangesSeq(SmallVec<[(u64, ChunkRanges); 2]>);
```

주요 특징:

- **런 길이 인코딩**: 반복되는 범위 패턴을 압축하여 저장
- **무한 시퀀스 지원**: 마지막 범위를 무한히 반복하는 패턴 지원
- **효율적인 와이어 인코딩**: postcard 직렬화를 통한 최적화된 네트워크 전송

불완전한 파일의 상태 관리

Bitfield를 통한 진행 상황 추적

```
pub struct Bitfield {
pub(crate) size: u64, // 파일의 총 크기
pub ranges: ChunkRanges, // 다운로드된 청크 범위
}
```

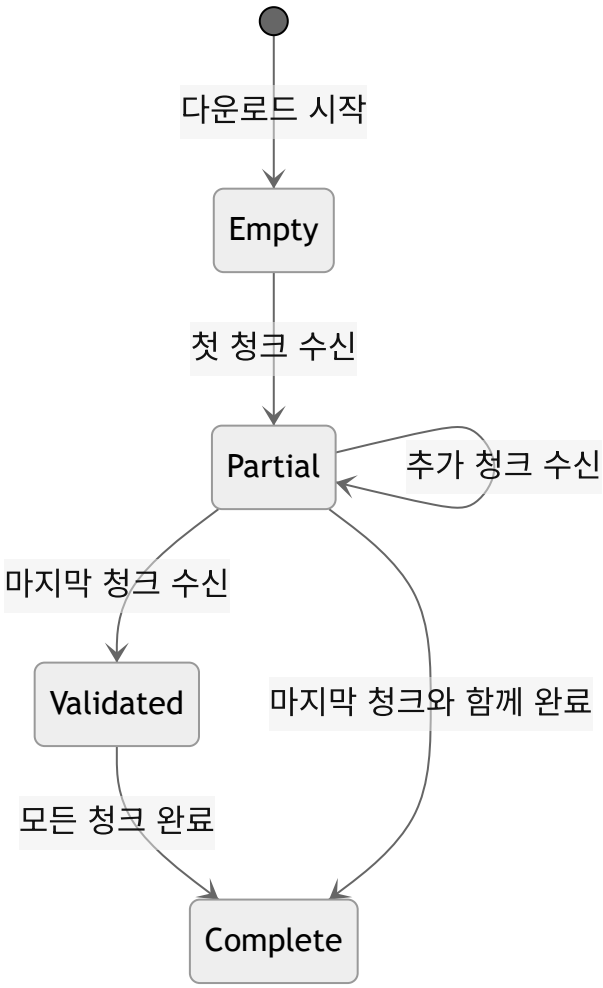
Bitfield 상태 분류:

1. **Empty**: 아무것도 다운로드되지 않은 상태
2. **Partial**: 일부 청크만 다운로드된 상태

3. **Validated**: 마지막 청크가 다운로드되어 크기가 검증된 상태

4. **Complete**: 모든 청크가 다운로드된 상태

상태 전환 과정



점진적 완성 과정

1. 청크 단위 검증

```
impl BaoFileStorage {
async fn write_batch(&mut self, batch: &[BaoContentItem], bitfield: &Bitfield
for item in batch {
match item {
```

```
BaoContentItem::Leaf(leaf) => {
// 데이터 청크 저장
self.data.write_at(leaf.offset, &leaf.data)?;
// 비트필드 업데이트
let  added = chunk_range(&leaf);
let  update = self.bitfield.update(&Bitfield::new(added, self.size));
// 완료 상태 확인
if  update.new_state().complete {
self.transition_to_complete();
}
}
BaoContentItem::Parent(parent) => {
// 해시 트리 노드 저장
self.outboard.save(parent.node, &parent.pair).await?;
}
}
}
Ok(())
}
}
```

2. 실시간 무결성 검증

- **스트리밍 검증**: 데이터 수신과 동시에 BLAKE3 해시 검증 수행

- **부분 검증**: 완전한 파일이 없어도 수신된 청크의 무결성 즉시 확인
- **트리 구조 활용**: BAO 트리를 통한 효율적인 부분 검증

비트필드를 이용한 다운로드 진행 상황 추적

비트필드 업데이트 메커니즘

```
pub enum UpdateResult {
NoChange(BitfieldState), // 변화 없음
MinorChange(BitfieldState), // 작은 변화 (새 청크 추가)
MajorChange(BitfieldState, BitfieldState), // 주요 변화 (상태 전환)
}

impl Bitfield {
pub fn update(&mut self, update: &Bitfield) -> UpdateResult {
let old_state = self.state();
let new_bitfield = self.clone().combine(update.clone());
if new_bitfield != *self {
let new_state = new_bitfield.state();
*self = new_bitfield;
if old_state != new_state {
UpdateResult::MajorChange(old_state, new_state)
} else {
UpdateResult::MinorChange(new_state)
```

```
}
} else {
UpdateResult::NoChange(old_state)
}
}
}
```

진행률 계산

1. 바이트 기반 진행률

```
impl Bitfield {
pub fn total_bytes(&self) -> u64 {
let mut total = 0;
for range in self.ranges.iter() {
let (start, end) = match range {
RangeSetRange::Range(range) => {
(range.start.to_bytes(), range.end.to_bytes().min(self.size))
}
RangeSetRange::RangeFrom(range) => (range.start.to_bytes(), self.size),
};
total += end - start;
}
total
```

```
}
pub fn progress_percentage(&self) -> f64 {
if self.size == 0 {
return 100.0;
}
(self.total_bytes() as f64 / self.size as f64) * 100.0
}
}
```

2. 청크 기반 진행률

```
impl Bitfield {
pub fn chunk_progress(&self) -> (u64, u64) {
let total_chunks = ChunkNum::chunks(self.size);
let completed_chunks = self.ranges.iter()
.map(|range| match range {
RangeSetRange::Range(r) => r.end.0 - r.start.0,
RangeSetRange::RangeFrom(r) => total_chunks - r.start.0,
})
.sum();
(completed_chunks, total_chunks)
}
```

```
}
```

관찰자 패턴을 통한 실시간 업데이트

```
pub struct BaoFileHandle {
state: Arc<watch::Sender<BaoFileStorage>>,
}

impl BaoFileHandle {
pub fn subscribe(&self) -> BitfieldSubscriber {
BitfieldSubscriber {
receiver: self.state.0.subscribe(),
}
}
}

pub struct BitfieldSubscriber {
receiver: watch::Receiver<BaoFileStorage>,
}

impl BitfieldSubscriber {
pub async fn forward(&mut self, mut tx: mpsc::Sender<Bitfield>) -> Result<(
while self.receiver.changed().await.is_ok() {
```

```
let bitfield = self.receiver.borrow().bitfield();
if tx.send(bitfield).await.is_err() {
break;
}
}
Ok(())
}
}
```

다중 소스로부터의 병렬 다운로드 처리

엔티티 매니저를 통한 동시성 제어

```
pub struct EntityManagerState<P: Params> {
entities: HashMap<P::EntityId, EntitySlot<P>>,
global_state: P::GlobalState,
max_concurrent: usize,
}

impl<P: Params> EntityManagerState<P> {
pub async fn spawn<F, Fut>(&mut self, id: P::EntityId, f: F) -> Option<JoinH
where
F: FnOnce(SpawnArg<P>) -> Fut + Send + 'static,
```

```
Fut: Future<Output = ()> + Send + 'static,
{
match  self.entities.entry(id) {
Entry::Occupied(mut  entry) => {
let  slot = entry.get_mut();
match  slot {
EntitySlot::Active(state) => {
// 이미 활성화된 엔티티 - 새 작업 스케줄링
Some(tokio::spawn(f(SpawnArg::Active(state.clone()))))
}
EntitySlot::Busy => {
// 바쁜 상태 - 대기열에 추가하거나 거부
Some(tokio::spawn(f(SpawnArg::Busy)))
}
}
}
Entry::Vacant(entry) => {
// 새 엔티티 생성
let  state = ActiveEntityState::new(id, self.global_state.clone());
entry.insert(EntitySlot::Active(state.clone()));
Some(tokio::spawn(f(SpawnArg::Active(state))))
}
}
```

```
}

}
```

해시별 동시성 제어

각 해시에 대해 하나의 활성 다운로드만 허용하되, 여러 클라이언트가 동일한 다운로드를 공유할 수 있습니다:

```
impl HashContext {

pub  async  fn  import_bao(&self, cmd: ImportBaoMsg) {

// 동일한 해시에 대한 다른 요청들과 상태 공유

let  mut  stream = cmd.rx;

while  let Some(item) = stream.recv().await {

// 모든 구독자에게 업데이트 브로드캐스트

self.write_batch(&[item], &bitfield).await?;

}

}

}
```

범위 병합 및 최적화

1. 중복 범위 제거

```
impl GetManyRequest {

pub  fn  builder() -> GetManyRequestBuilder {

GetManyRequestBuilder::default()
```

```
}
}

impl GetManyRequestBuilder {
pub fn hash(mut self, hash: Hash, ranges: ChunkRanges) -> Self {
// 동일한 해시에 대한 범위들을 자동으로 병합
match self.ranges.entry(hash) {
Entry::Occupied(mut entry) => {
*entry.get_mut() |= ranges; // 범위 합집합
}
Entry::Vacant(entry) => {
entry.insert(ranges);
}
}
self
}
}
```

2. 효율적인 범위 분할

```
pub fn optimize_ranges(requested: &ChunkRanges, available: &ChunkRanges) -> Ve
let missing = requested - available; // 아직 다운로드되지 않은 범위
// 큰 연속 범위를 우선적으로 처리
```

```
missing.iter()
.filter_map(|range| match range {
RangeSetRange::Range(r) if r.end.0 - r.start.0 > 16 => Some(r.clone().into()),
_ => None,
})
.collect()
}
```

다중 소스 조정

1. 소스별 성능 추적

```
pub struct SourceStats {
pub bytes_per_second: f64,
pub error_rate: f64,
pub last_seen: Instant,
}

pub struct MultiSourceDownloader {
sources: HashMap<NodeId, SourceStats>,
active_downloads: HashMap<ChunkRanges, NodeId>,
}

impl MultiSourceDownloader {
```

```
pub fn select_best_source(&self, range: &ChunkRanges) -> Option<NodeId> {
self.sources.iter()
.filter(|(_, stats)| stats.error_rate < 0.1) // 에러율 10% 미만
.max_by(|(_, a), (_, b)| a.bytes_per_second.partial_cmp(&b.bytes_per_second).unw
.map(|(node_id, _)| *node_id)
}
}
```

2. 동적 범위 재할당

```
impl MultiSourceDownloader {
pub async fn rebalance_ranges(&mut self) {
for (range, source) in &self.active_downloads {
let stats = &self.sources[source];
// 느린 소스에서 빠른 소스로 범위 재할당
if stats.bytes_per_second < self.average_speed() * 0.5 {
if let Some(better_source) = self.select_best_source(range) {
self.reassign_range(range.clone(), better_source).await;
}
}
}
}
```

```
}
```

성능 최적화 기법

1. 청크 그룹 최적화

- **16KiB 청크 그룹**: 16개의 1024바이트 청크를 하나의 그룹으로 처리
- **메타데이터 오버헤드 감소**: 청크 그룹 단위로 해시 트리 노드 전송
- **I/O 효율성**: 큰 단위의 연속 읽기/쓰기 작업

2. 메모리 사용량 최적화

- **스트리밍 처리**: 전체 파일을 메모리에 로드하지 않고 청크 단위로 처리
- **참조 카운팅**: 동일한 데이터에 대한 여러 참조 시 메모리 공유
- **임계값 기반 저장**: 작은 파일은 메모리, 큰 파일은 디스크에 저장

3. 네트워크 최적화

- **파이프라이닝**: 여러 청크 요청을 동시에 전송
- **적응적 윈도우 크기**: 네트워크 상태에 따른 동시 요청 수 조정
- **연결 재사용**: QUIC 연결을 여러 요청에서 공유

오류 처리 및 복구 전략

1. 부분 전송 실패 처리

```
pub  enum DecodeError {

ChunkNotFound {}, // 청크를 찾을 수 없음
```

```
ParentNotFound { node: TreeNode }, // 부모 노드를 찾을 수 없음
LeafNotFound { num: ChunkNum }, // 리프 노드를 찾을 수 없음
ParentHashMismatch { node: TreeNode }, // 부모 해시 불일치
LeafHashMismatch { num: ChunkNum }, // 리프 해시 불일치
}
```

2. 재시도 메커니즘

```
impl PartialDownloadManager {
pub  async  fn  retry_failed_chunks(&mut  self, failed_chunks: ChunkRanges) -> R
let  mut  retry_count = 0;
let  max_retries = 3;
while  retry_count < max_retries && !failed_chunks.is_empty() {
match  self.download_chunks(failed_chunks.clone()).await {
Ok(_) => break,
Err(e) => {
retry_count += 1;
let  delay = Duration::from_secs(2_u64.pow(retry_count));
tokio::time::sleep(delay).await;
}
}
}
Ok(())
```

```
}

}
```

3. 상태 복구

```
impl BaoFileStorage {

pub  async  fn  recover_from_corruption(&mut  self, corrupted_range: ChunkRanges

// 손상된 범위를 비트필드에서 제거

self.bitfield.ranges = &self.bitfield.ranges - &corrupted_range;

// 해당 범위의 데이터 삭제

for  range  in  corrupted_range.iter() {

match  range {

RangeSetRange::Range(r) => {

let  start = r.start.to_bytes();

let  end = r.end.to_bytes();

self.data.clear_range(start..end)?;

}

_ => {}

}

}

// 재다운로드 요청

self.request_redownload(corrupted_range).await
```

```
}

}
```

실제 사용 예제

1. 대용량 파일의 부분 다운로드

```
async fn download_file_range(

connection: Connection,

hash: Hash,

start_byte: u64,

end_byte: u64,

) -> Result<Vec<u8>> {

let request = GetRequest::builder()

.root(ChunkRanges::bytes(start_byte..end_byte))

.build(hash);

let mut fsm = fsm::start(connection, request, RequestCounters::default());

let connected = fsm.next().await?;

let (header, _size) = connected.next().await?.next().await?;

let (end_blob, data) = header.concatenate_into_vec().await?;

let _stats = end_blob.next().next().await?;

Ok(data)
```

```
}
```

2. 진행 상황 모니터링

```
async fn monitor_download_progress(hash: Hash, store: &MemStore) -> Result<()>
let mut observer = store.observe(hash).await?;
while let Some(bitfield) = observer.recv().await {
let progress = bitfield.progress_percentage();
let (completed_chunks, total_chunks) = bitfield.chunk_progress();
println!("Progress: {:.1}% ({}/{} chunks)",
progress, completed_chunks, total_chunks);
if bitfield.is_complete() {
println!("Download completed!");
break;
}
}
Ok(())
}
```

3. 다중 소스 다운로드

```
async fn download_from_multiple_sources(
hash: Hash,
```

```
sources: Vec<NodeId>,
endpoint: &Endpoint,
) -> Result<()> {
let mut tasks = JoinSet::new();
let total_size = get_blob_size(hash, &sources[0], endpoint).await?;
let chunk_count = ChunkNum::chunks(total_size);
let chunks_per_source = chunk_count / sources.len() as u64;
for (i, source) in sources.iter().enumerate() {
let start_chunk = i as u64 * chunks_per_source;
let end_chunk = if i == sources.len() - 1 {
chunk_count
} else {
(i as u64 + 1) * chunks_per_source
};
let range = ChunkRanges::chunks(start_chunk..end_chunk);
let request = GetRequest::blob_ranges(hash, range);
tasks.spawn(download_range(*source, request, endpoint.clone()));
}
while let Some(result) = tasks.join_next().await {
result??;
}
Ok(())
```

```
}
```

결론

iroh-blob의 부분 다운로드 및 범위 요청 시스템은 다음과 같은 핵심 특징을 제공합니다:

1. **효율적인 범위 표현**: ChunkRanges를 통한 압축된 범위 표현과 불연속 범위 지원
2. **실시간 진행 추적**: Bitfield를 통한 세밀한 다운로드 상태 관리
3. **강력한 무결성 보장**: BLAKE3 기반의 스트리밍 검증으로 부분 데이터도 안전하게 검증
4. **다중 소스 지원**: 여러 소스로부터의 병렬 다운로드와 동적 로드 밸런싱
5. **복구 메커니즘**: 네트워크 중단이나 데이터 손상 시 효율적인 복구 전략

이러한 기능들이 결합되어 대용량 파일의 안정적이고 효율적인 분산 전송을 가능하게 합니다.